

操作系统

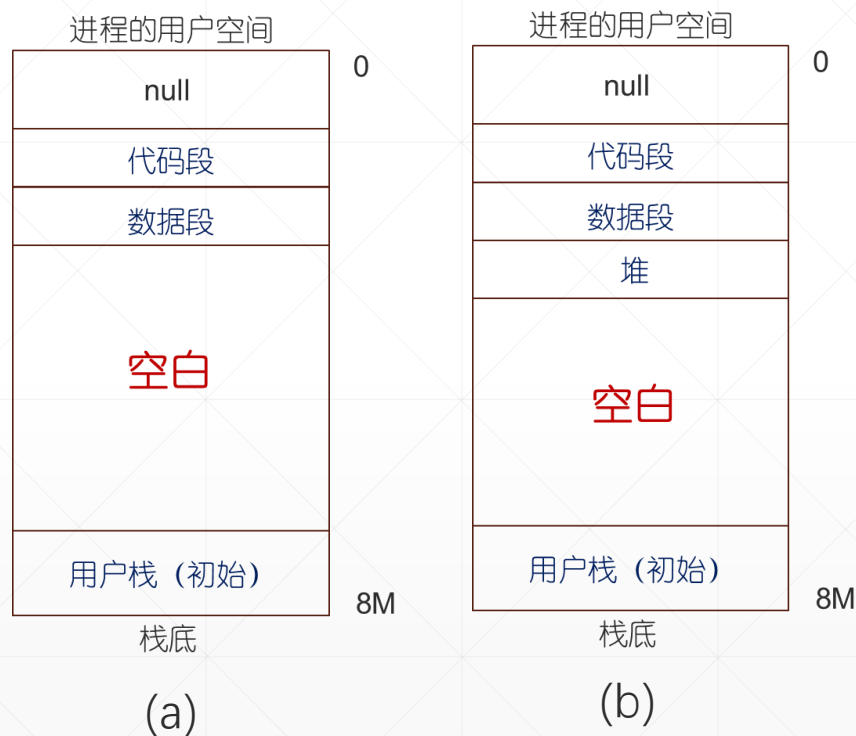
第二章 进程管理

Unix V6 + 用户堆管理 malloc 和 free



Part 1、用户空间的堆 和 动态内存分配器

堆、malloc 和 free



- exec返回时，应用程序从main()的入口开始执行。进程图像没有堆。图 (a)。
- 应用程序执行malloc()函数后，进程启用堆空间，存放应用程序申请的动态内存。图 (b)。
- 堆开始于数据段之后的第1个字节。
 - 堆，动态变化，向高地址方向延伸。执行malloc() 堆扩大；执行free()堆缩小。
- 栈开始于用户空间最高端，Unix V6++是8M。
 - 栈，动态变化，向低地址方向延伸。子程序调用栈扩大，子程序返回栈缩小。
- 堆的底部 和 栈顶之间的空白空间，堆栈共享。堆和栈头顶头的时候，OOM (Out Of Memory)，虚空间用完了，进程挂掉。

堆空间管理，动态内存分配器

- 堆空间是用户态的，由库函数管理。Unix V6++ 是 lib/malloc.c 的 malloc() 和 free()，这是 Unix 的动态内存分配器。
- 堆空间不足，malloc 执行 sbrk 系统调用，向内核申请、追加堆空间。Unix V6++，一次 3 页。这是进程图像扩展。
- 堆空间太多浪费，free 执行 sbrk 系统调用，缩小进程图像。Unix V6++，一次 6 页。
- 动态内存分配器维护 2 个全局变量，malloc_begin 是堆空间的起始地址，malloc_end 是堆空间之后，第一个字节的地址。



内存片

- 内存片是堆空间中一块已分配内存区，包括1个flist结构和1块动态内存。

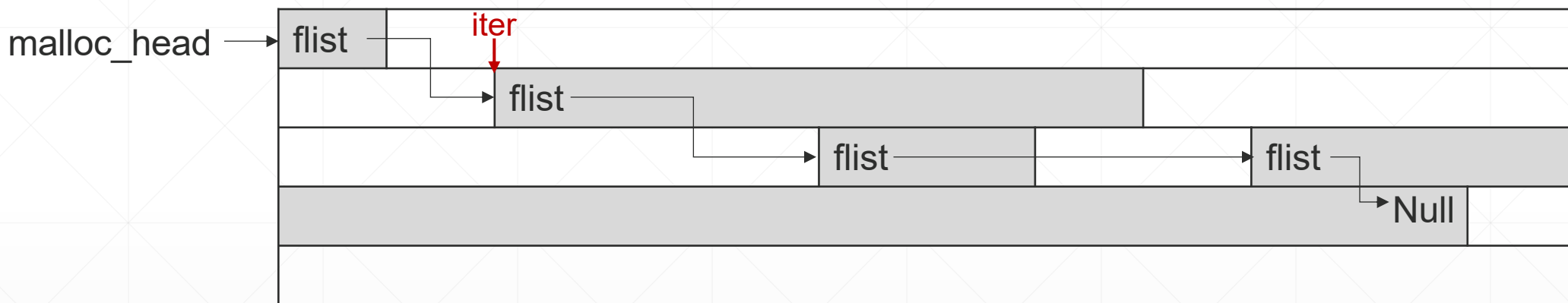


```
typedef struct flist {  
    unsigned int size; // 长度  
    struct flist *nlink; // 下个内存片的首地址  
};
```

内存片链表

- 堆空间，内存片、空闲区交错。

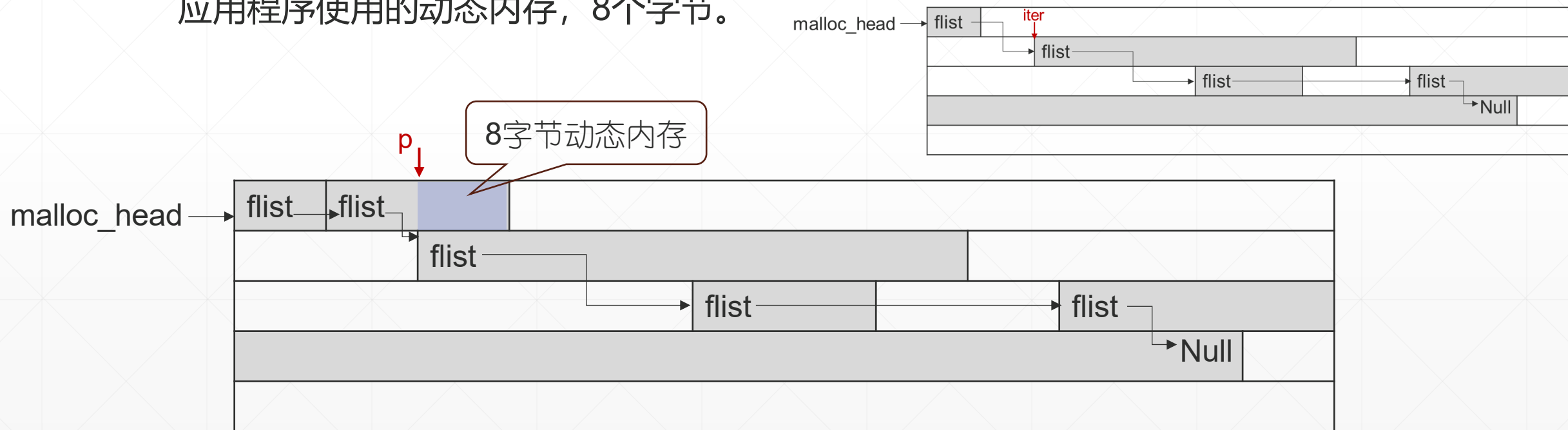
所有内存片，按首地址形成单向链表。



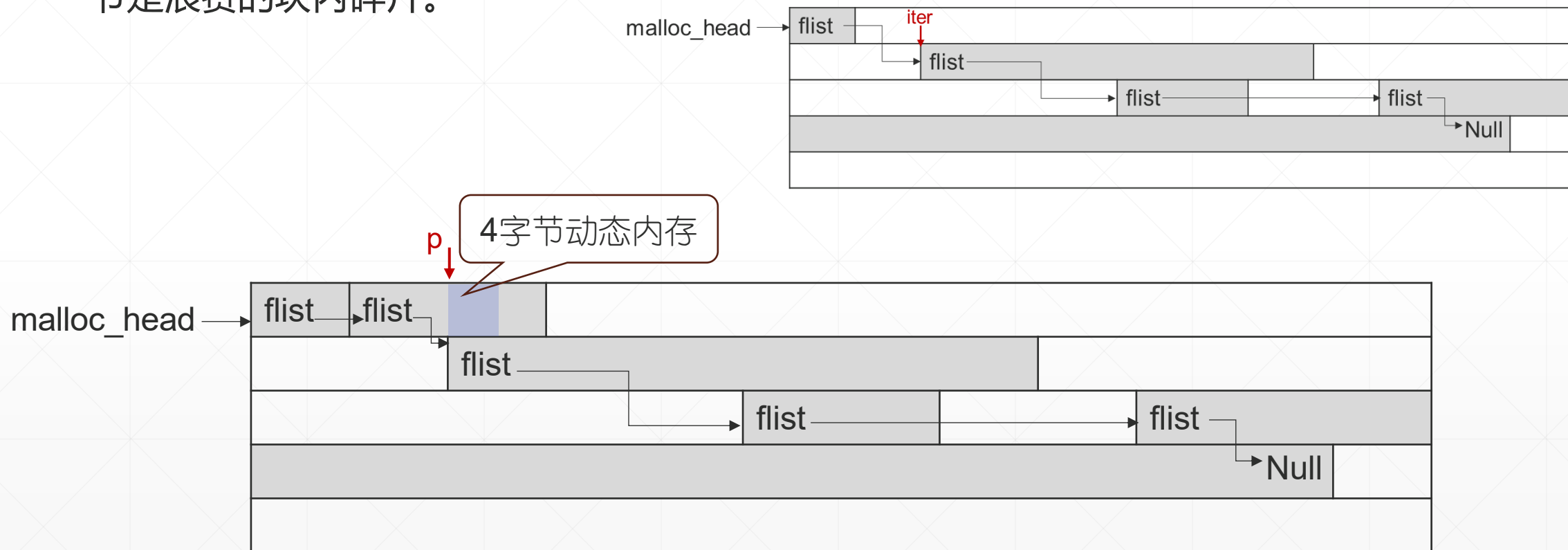
- 如图，一个堆，4个内存片，4个空闲区。全局变量malloc_head是内存片链表的首部。
- 首地址为iter的内存片，其后空闲区，尺寸为 $(\text{int})(\text{iter} \rightarrow \text{nlink}) - [(\text{int})\text{iter} + \text{iter} \rightarrow \text{size}]$
- 链表中的第一个内存片是哑元（dump），8字节，只有flist，没有动态内存。

动态内存分配, malloc(size)

- malloc() 寻找尺寸大于size+8的空闲区。构造内存片。维护链表。
- 例 1: $p = \text{malloc}(8)$ 。产生的内存片尺寸是16字节，其中淡蓝色阴影处是分配给应用程序使用的动态内存，8个字节。



- 例 2: 回到PPT6的状态, $p = \text{malloc}(4)$ 。产生的内存片尺寸依然是16字节, 其中 flist 8字节; 淡蓝色阴影处是分配给应用程序使用的动态内存, 4字节; 最后的4字节是浪费的块内碎片。



源代码 1: 库函数 malloc()

搜索内存片链表，找足够大的空闲区 temp

lib/src/malloc.c

```
1#include <sys.h>
2#include <malloc.h>
3#include <stdio.h>
4
5#define PAGE_SIZE 12288
6
7char *malloc_begin = NULL;
8char *malloc_end = NULL;
9
10typedef struct flist {
11    unsigned int size;
12    struct flist *nlink;
13};
14
15struct flist *malloc head = NULL;
```

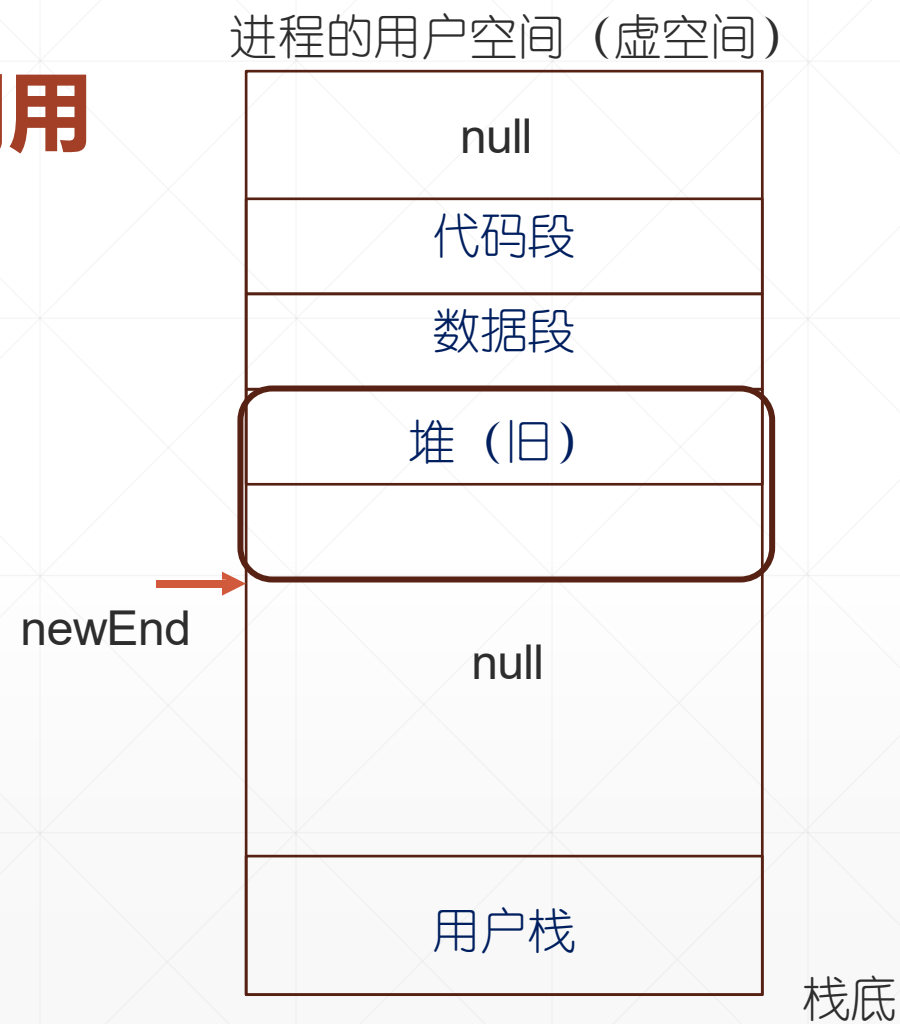
```
17void* malloc(unsigned int size)
18{
19    if (malloc_begin == NULL)
20    { // 1、首次执行malloc, 启用堆
21        malloc_begin = sbrk(0);
22        malloc_end = sbrk(PAGE_SIZE); //3页
23        malloc_head = malloc_begin;
24        malloc_head->size = sizeof(struct flist);
25        malloc_head->nlink = NULL;
26    }
27    if (size == 0)
28    {
29        return NULL;
30    }
31    size += sizeof(struct flist);
32    size = ((size + 7) >> 3) << 3;
33    struct flist* iter = malloc head;
```



```
35 // 2、分配动态内存（找一个足够大的空闲片）
36 struct flist *temp;
37 while( iter->nlink != NULL )
38 { // 搜索链表
39     if ((int)(iter->nlink) - iter->size - (int)iter >= size)
40     {
41         temp = (char *)iter + (iter->size);
42         temp->nlink = iter->nlink;
43         iter->nlink = temp;
44         temp->size = size;
45         return (char *)temp + sizeof(struct flist);
46     }
47     iter = iter->nlink;
48 }
49
50 int remain;
51 L1: remain = malloc_end - iter->size - (int)iter;
52 if ( remain >= size)
53 { // 最后一个内存片之后的空间，足够吗？
54     temp = (char *)iter + (iter->size);
55     temp->nlink = NULL;
56     iter->nlink = temp;
57     temp->size = size;
58     return (char *)temp + sizeof(struct flist);
59 }
60
61 // 3、内存分配不成功，执行sbrk系统调用，扩展堆空间
62 int expand = size - remain;
63 expand = ((expand + PAGE_SIZE - 1) / PAGE_SIZE) * PAGE_SIZE; // 扩展3页的整数倍
64 malloc_end = sbrk(expand);
65 goto L1; // 分配
66 }
```

Part 2、改变堆空间的尺寸，brk系统调用

- `brk(0)`，返回当前堆的尾部地址
- `brk(newEnd)`，改变堆空间尺寸。`newEnd`是新堆的尾部地址



动态内存分配器使用brk系统调用的包装函数， sbrk(increment)

- 动态内存分配器维护一个全局变量 fakeedata，跟踪堆尾部
- 用函数 sbrk(increment) 扩展 或 缩小 堆空间。入口参数 increment，堆尺寸的变化量。
 - 扩大，正值；缩小，负值

进程的用户空间（虚空间）





源代码 2: sbrk(increment)

unsigned int fakeedata = 0; // 堆底部（其后，第一个字节的地址）。静态变量，函数返回，值保留。

```
int sbrk(int increment)
```

```
{
```

```
    if (fakeedata == 0) // 首次调用sbrk函数，初始化fakeedata
```

```
    {
```

```
        fakeedata = brk(0); // 执行brk(0)得到数据段底部地址
```

```
    }
```

```
    unsigned int newedata = fakeedata + increment - 1; // 1、计算新数据段最后一个字节的地址
```

```
    brk(((newedata >> 12) + 1) << 12); // 2、执行brk系统调用，改变数据段的尺寸（以页为单位）
```

```
    fakeedata = newedata + 1;
```

```
    return fakeedata; // 3、返回新数据段底部
```

```
}
```

源代码 3: brk系统调用

SBreak(), brk系统调用的处理函数。

按新尺寸重写内存描述符 和 相对虚实地址映射表,
调用Expand()函数为新的可交换部分分配物理内存[注]。

```
int SystemCall::Sys_SBreak()      入口函数
{
    User& u = Kernel::Instance().GetUser();
    u.u_procp->SBreak();
    return 0; /* GCC likes it ! */
}
```

u_arg[0], 新数据段之后的第一个字节的地址。

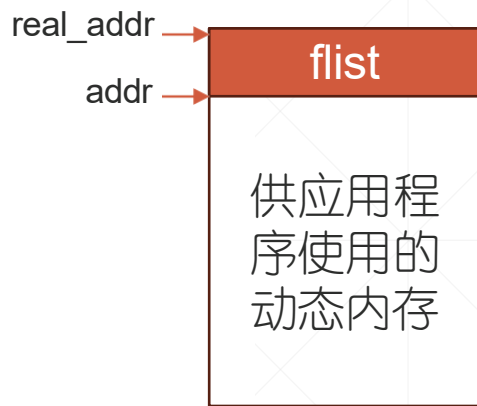
[注] 现代操作系统广泛使用基于请求调页的虚拟内存技术, 所以进程图像扩展时, brk系统调用只更新进程虚地址空间, 不需要为新空间分配物理内存, 更无需移动已有进程图像。Unix V6++系统使用连续内存管理方式, brk系统调用需要在更新进程虚地址空间的同时, 同步更新分配给进程的物理内存单元。



源代码 4: 库函数 free()

lib/src/malloc.c

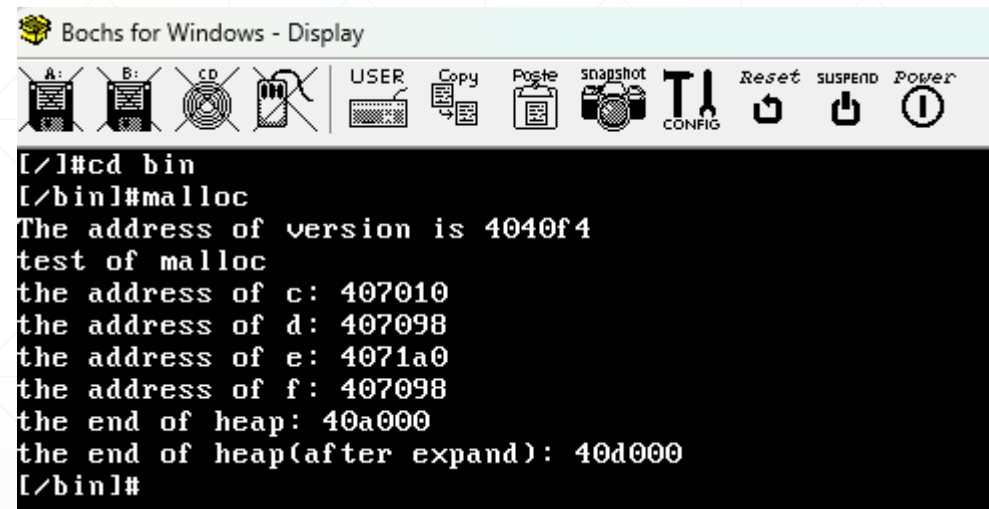
```
68 int free(void* addr) // 释放以addr为首地址的动态内存
69 {
70     char * real_addr = addr - 8; // 1、定位释放内存片首地址
71     struct flist* iter = malloc_head;
72     struct flist* last; // 用来定位释放内存片之前的内存片
73     if (addr == 0)
74     {
75         return -1;
76     }
77
78     while(iter) // 遍历链表
79     {
80         if (iter == real_addr) // 2、找到释放的内存片 iter
81         {
82             last->nlink = iter->nlink; // 把 iter 从链表里删除
83             if (last->nlink == NULL) // 3、如果被删除的是最后一个内存片
84             {
85                 char *pos = (char *)last + last->size;
86                 if (malloc_end - pos > PAGE_SIZE * 2)
87                 { // 若删除后，堆空间尾部出现尺寸大于6页的空闲片，sbrk缩小堆空间
88                     malloc_end = sbrk(-(malloc_end - pos) / PAGE_SIZE * PAGE_SIZE);
89                 }
90             }
91             return 0;
92         }
93         last = iter; // last和 iter 同步后移
94         iter = iter->nlink;
95     }
96     return -1;
97 }
```



源代码 5：测试动态内存分配器的应用程序

program/malloc.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <malloc.h>
4 #include <sys.h>
5
6 int version = 1;
7
8 int main1()
9 {
10     int i, j;
11     printf("The address of version is %x\n", &version);
12
13     printf("test of malloc\n");
14     int *c = malloc(128);
15     printf("the address of c: %x\n", c);
16     int *d = malloc(256);
17     printf("the address of d: %x\n", d);
18     int *e = malloc(256);
19     printf("the address of e: %x\n", e);
20     free(d);
21     int *f = malloc(83);
22     printf("the address of f: %x\n", f);
23
24     printf("the end of heap: %x\n", sbrk(0));
25
26     f = malloc(3*4096);
27     printf("the end of heap(after another expand): %x\n", sbrk(0));
28
29     return 0;
30 }
```



```
[C:\]#cd bin
[C:\bin]#malloc
The address of version is 4040f4
test of malloc
the address of c: 407010
the address of d: 407098
the address of e: 4071a0
the address of f: 407098
the end of heap: 40a000
the end of heap(after another expand): 40d000
[C:\bin]#
```